

Szerződések CRUD műveletei

Szerződés létrehozása:

A szerződés létrehozása funkció fejlesztése során elkészítettem azt az API végpontot, amely új bérleti szerződések rögzítését teszi lehetővé az adatbázisban. A végpont fogadja az ingatlan és a bérelő azonosítóját, valamint a szerződéshez kapcsolódó adatokat, például a kezdő- és végdátumot, a havi díjat és a státuszt. A fejlesztés során SQL INSERT utasítással valósítottam meg az adatok eltárolását. A sikeres művelet után a rendszer visszajelzést küld a felhasználó számára. Az elkészült funkció működését Postman alkalmazással teszteltem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    ingatlanid,
    berloid,
    kezdetdatum,
    vegdatum,
    havidi,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO szerzodesek
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidi, statusz)
      VALUES (?, ?, ?, ?, ?, ?)` ,
      [
        ingatlanid,
        berloid,
        kezdetdatum,
        vegdatum,
        havidi,
        statusz
      ]
    );

    res.json({
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        siker: true,
        uzenet: "Szerződés sikeresen létrehozva!"
    });

} catch (err) {

    res.status(500).json(err);

} finally {

    if(conn) conn.release();

}

});

module.exports = router;
```

The screenshot shows the Postman interface for a POST request to `http://localhost:3001/api/szerzodesletrehoz`. The request body is a JSON object: `{ "havidij": 180000, "statusz": "Aktiv" }`. The response is a JSON object: `{ "siker": true, "uzenet": "Szerződés sikeresen létrehozva!" }`. The status is 200 OK, 8 ms, 324 B.

1.Részlet a szerzodesletrehoz.js fájlból.

Szerződés módosítása:

A szerződés módosítása funkció elkészítése során lehetővé tettem a meglévő szerződések adatainak frissítését. A rendszer képes módosítani a szerződés kezdő- és végdátumát, havi díját és státuszát. A fejlesztés során SQL UPDATE lekérdezést alkalmaztam az adatok frissítésére. A sikeres módosítás után a rendszer visszaigazoló üzenetet küld. A működés helyességét Postman alkalmazással teszteltem.

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    kezdetdatum,
    vegdatum,
    havidij,
    statusz
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE szerzodesek
      SET
      kezdetdatum = ?,
      vegdatum = ?,
      havidij = ?,
      statusz = ?
      WHERE id = ?`,
      [
        kezdetdatum,
        vegdatum,
        havidij,
        statusz,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes modositva!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

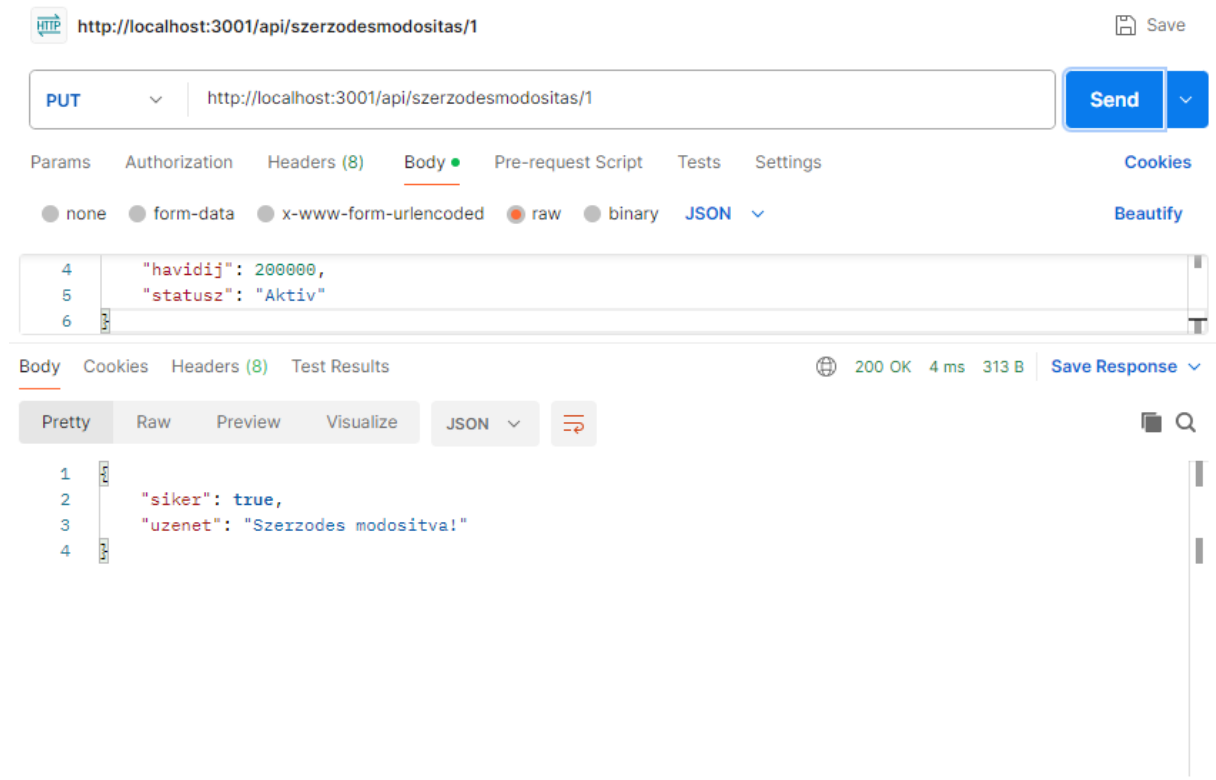
Orosz Kristóf – EYZWG9

```
        if(conn) conn.release();

    }

});

module.exports = router;
```



2.Részlet a szerzodesmodositas.js fájlból.

Szerződés törlése:

A szerződés törlése funkció fejlesztése során elkészítettem a szerződések eltávolítására szolgáló API végpontot. A rendszer a megadott azonosító alapján törli a kiválasztott szerződést az adatbázisból. A fejlesztés során figyelmet fordítottam a hibakezelésre és az adatbázis-kapcsolat megfelelő lezárására. A sikeres törlés után a rendszer visszajelzést küld a felhasználónak. Az elkészült funkció működését Postman segítségével ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

    let conn;

    try {
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM szerzodesek WHERE id = ?",
      [req.params.id]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes torolve!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```

The screenshot shows a REST client interface with the following details:

- URL: `http://localhost:3001/api/szerzodesstorles/1`
- Method: `DELETE`
- Response Status: `200 OK`, 4 ms, 311 B
- Response Body (JSON):

```
{  \"siker\": true,  \"uzenet\": \"Szerzodes torolve!\"}
```

3.Részlet a szerzodesstorles.js fájlból.

Szerződés listázása:

A szerződések listázása funkció fejlesztése során elkészítettem az összes rögzített szerződés lekérdezésére szolgáló API végpontot. A rendszer az adatbázisból lekéri a szerződések adatait és JSON formátumban továbbítja azokat a kliens számára. A fejlesztés célja az volt, hogy a felhasználók egyszerűen áttekinthető formában láthassák a szerződéseket. A lekérdezés során biztosítottam az adatbázis-kapcsolat megfelelő kezelését. A funkció helyes működését Postman segítségével ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    const adatok = await conn.query(
      "SELECT * FROM szerzodesek"
    );

    res.json(adatok);

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

The screenshot shows a web browser's developer tools interface. At the top, the address bar displays the URL `http://localhost:3001/api/szerzodeslistazas`. Below the address bar, the 'GET' method is selected, and the 'Send' button is visible. The 'Body' tab is active, showing the request body as a JSON object: `{"havidij": 180000, "statusz": "Aktiv"}`. The 'Test Results' tab is also visible, showing a status of '200 OK' with a response time of '5 ms' and a size of '580 B'. The 'Save Response' button is present. The 'Body' tab is further expanded, showing the response body as a JSON array of two objects. The first object has the following properties: `"id": 4, "ingatlanid": 3, "berloid": 2, "kezdetdatum": "2025-12-31T23:00:00.000Z", "vegdatum": "2026-12-30T23:00:00.000Z", "havidij": "180000.00", "statusz": "Aktiv"`. The second object has the property `"id": 5,`.

```
1 {
2   {
3     "id": 4,
4     "ingatlanid": 3,
5     "berloid": 2,
6     "kezdetdatum": "2025-12-31T23:00:00.000Z",
7     "vegdatum": "2026-12-30T23:00:00.000Z",
8     "havidij": "180000.00",
9     "statusz": "Aktiv"
10  },
11  {
12    "id": 5,
```

4.Részlet a szerzodeslistazas.js fájlból.